AWS Lambda For JAVA Developers Assignment Solutions

Assignment 1 : Parameters And Return types

```
package helloworld;

import java.time.LocalDateTime;

import java.util.Map;

import helloworld.dto.Payment;

import helloworld.dto.Ticket;

public class App

{

public void truckTracker(Map<String, Double> coordinates) {

System.out.println("Longitude: " + coordinates.get("longitude") + " Latitude:
" + coordinates.get("latitiude"));

}

public Ticket getTicket(Payment payment) {

if (Float.compare(payment.getAmount(), Ticket.price) < 0) {

throw new RuntimeException("Insufficient payment.");
```

```
}
        return new Ticket("Avengers End Game", LocalDateTime.now());
}
}
```

Assignment 2 : Create Serverless API

```
        public class CreateCustomerLambda {
        private ObjectMapper objectMapper;
        private DynamoDB dynamoDB;
        private Table customersTable;
        public CreateCustomerLambda() {
        objectMapper = new ObjectMapper();
        dynamoDB = new
DynamoDB(AmazonDynamoDBClientBuilder.defaultClient());
        customersTable =
dynamoDB.getTable(System.getenv("CUSTOMERS_TABLE"));
}
        public APIGatewayProxyResponseEvent
createCustomer(APIGatewayProxyRequestEvent request) throws Exception {
```

```
        Customer customer = objectMapper.readValue(request.getBody(),
Customer.class);

        Item item = new Item().withPrimaryKey("id", customer.getId())

        .withString("firstName", customer.getFirstName())

        .withString("lastName", customer.getLastName())

        .withInt("rewardPoints", customer.getRewardPoints());

        customersTable.putItem(item);

        return new
APIGatewayProxyResponseEvent().withStatusCode(200).withBody(customer.get
Id().toString());

}

}

        public class ReadCustomersLambda {

        private ObjectMapper objectMapper;

        private AmazonDynamoDB amazonDynamoDB;

        public ReadCustomersLambda() {

        objectMapper = new ObjectMapper();

        amazonDynamoDB = AmazonDynamoDBClientBuilder.defaultClient();

}

        public APIGatewayProxyResponseEvent
```

```
getOrders(APIGatewayProxyRequestEvent request) throws Exception {

        ScanResult scanResult = amazonDynamoDB.scan(new
ScanRequest().withTableName(System.getenv("CUSTOMERS_TABLE")));

        List<Customer> orders = scanResult

        .getItems()

        .stream()

        .map(item -> new Customer(Long.parseLong(item.get("id").getN()),

        item.get("firstName").getS(),

        item.get("lastName").getS(),

        Integer.parseInt(item.get("rewardPoints").getN())))

        .collect(Collectors.toList());

        return new
APIGatewayProxyResponseEvent().withStatusCode(200).withBody(objectMappe
r.writeValueAsString(orders));

}

}
```

Assignment 3 : S3 Event as Trigger

```
        public class ReadStudentLambda {

        private final AmazonS3 amazonS3 =
```

```
AmazonS3ClientBuilder.defaultClient();

        private final ObjectMapper objectMapper = new ObjectMapper();

        private final AmazonSNS amazonSNS =
AmazonSNSClientBuilder.defaultClient();

        public void handler(S3Event s3Event) {

        s3Event.getRecords().forEach(record -> {

        S3ObjectInputStream s3ObjectInputStream = amazonS3

        .getObject(record.getS3().getBucket().getName(),
record.getS3().getObject().getKey())

        .getObjectContent();

        try {

        List<Student> students =
Arrays.asList(objectMapper.readValue(s3ObjectInputStream, Student[].class));

        System.out.println(students);

        students.forEach(student -> {

        student.calculateGradeFromScore();

        try {

        amazonSNS.publish(System.getenv("PATIENT_CHECKOUT_TOPIC"),

        objectMapper.writeValueAsString(student));

        } catch (JsonProcessingException e) {
```

```
        e.printStackTrace();
}
});
        } catch (IOException e) {
        e.printStackTrace();
}
});
}
}
        public class ReportGeneratorLambda {
        private final ObjectMapper objectMapper = new ObjectMapper();
        public void handler(SNSEvent snsEvent) {
        snsEvent.getRecords().forEach(record -> {
        try {
        Student students =
objectMapper.readValue(record.getSNS().getMessage(),
        Student.class);
        System.out.println(students);
```

```
        } catch (JsonProcessingException e) {

        e.printStackTrace();

}

});

}

}
```

Assignment 4 : Logging

```
        public class ReadStudentLambda {

        private final AmazonS3 amazonS3 =
AmazonS3ClientBuilder.defaultClient();

        private final ObjectMapper objectMapper = new ObjectMapper();

        private final AmazonSNS amazonSNS =
AmazonSNSClientBuilder.defaultClient();

        public void handler(S3Event s3Event) {

        Logger logger = LoggerFactory.getLogger(ReadStudentLambda.class);

        s3Event.getRecords().forEach(record -> {

        S3ObjectInputStream s3ObjectInputStream = amazonS3

        .getObject(record.getS3().getBucket().getName(),
record.getS3().getObject().getKey())
```

```
        .getObjectContent();

        try {

        List<Student> students =
Arrays.asList(objectMapper.readValue(s3ObjectInputStream, Student[].class));

        logger.info(students.toString());

        students.forEach(student -> {

        student.calculateGradeFromScore();

        try {

        amazonSNS.publish(System.getenv("PATIENT_CHECKOUT_TOPIC"),

        objectMapper.writeValueAsString(student));

        } catch (JsonProcessingException e) {

        e.printStackTrace();

}

});

        } catch (IOException e) {

        logger.error(e.getMessage(), e);

}

});
```

```
}

}

        public class ReportGeneratorLambda {

        private final ObjectMapper objectMapper = new ObjectMapper();

        public void handler(SNSEvent snsEvent) {

        Logger logger = LoggerFactory.getLogger(ReadStudentLambda.class);

        snsEvent.getRecords().forEach(record -> {

        try {

        Student students =
objectMapper.readValue(record.getSNS().getMessage(),

        Student.class);

        logger.info(students.toString());

        } catch (JsonProcessingException e) {

        logger.error(e.getMessage(), e);

}

});

}

}
```

Assignment 5 : Error Handling

```
public class ReadStudentLambda {

private final AmazonS3 amazonS3 =
AmazonS3ClientBuilder.defaultClient();

private final ObjectMapper objectMapper = new ObjectMapper();

private final AmazonSNS amazonSNS =
AmazonSNSClientBuilder.defaultClient();

public void handler(S3Event s3Event) {

Logger logger = LoggerFactory.getLogger(ReadStudentLambda.class);

s3Event.getRecords().forEach(record -> {

S3ObjectInputStream s3ObjectInputStream = amazonS3

.getObject(record.getS3().getBucket().getName(),
record.getS3().getObject().getKey())

.getObjectContent();

try {

List<Student> students =
Arrays.asList(objectMapper.readValue(s3ObjectInputStream, Student[].class));

logger.info(students.toString());

students.forEach(student -> {

student.calculateGradeFromScore();
```

```
        try {
        amazonSNS.publish(System.getenv("PATIENT_CHECKOUT_TOPIC"),
        objectMapper.writeValueAsString(student));
        } catch (JsonProcessingException e) {
        logger.error(e.getMessage(), e);
        throw new RuntimeException(e);
}
});
        } catch (IOException e) {
        logger.error(e.getMessage(), e);
        throw new RuntimeException(e);
}
});
}
        public class ErrorHandler {
        public void handler(SNSEvent snsEvent, Context context) {
        LambdaLogger logger = context.getLogger();
        snsEvent.getRecords().forEach(record -> {
```

```
        logger.log("Dead Letter Queue Event: " + record.getSNS().getMessage());
});
}
}
```